Versiones y estructura de los mensajes HTTP — versión ampliada

Para leer una petición y una respuesta durante la clase, empieza por la versión corta. Esta guía explica con más detalle el vocabulario de transporte y ofrece ejemplos adicionales.

HTTP define la conversación entre clientes y servidores. Una petición pregunta o solicita una acción; una respuesta comunica qué ha ocurrido y puede devolver una representación.

La versión de HTTP cambia la forma de transportar los mensajes, pero mantiene la misma semántica básica: método, destino, estado, cabeceras y contenido.

Qué hay que entender primero

Para leer una traza HTTP no hace falta implementar TCP, UDP, QUIC ni frames binarios. Sí hay que distinguir estas piezas:

- **Petición:** mensaje que envía el cliente.
- **Respuesta:** mensaje que devuelve el servidor.
- **Método:** intención de la petición (`GET`, `POST`, `DELETE`…).
- **Destino:** ruta o recurso solicitado.
- **Estado:** resultado de la respuesta (`200`, `404`, `500`…).
- **Cabeceras:** metadatos del intercambio.
- **Cuerpo:** contenido opcional del mensaje.

Si el alumnado domina esa anatomía, después puede entender métodos, estados, cabeceras, caché, JSON y APIs REST con mucha menos confusión.

Versiones HTTP

HTTP/1.1

HTTP/1.1 se suele explicar con mensajes legibles como texto. Es ideal para aprender porque se ve claramente la estructura:

```
GET /api/hello HTTP/1.1
Host: localhost:8080
Accept: application/json
```

Normalmente usa TCP. Introdujo semántica web moderna y conexiones persistentes, entre otras mejoras.

HTTP/2

HTTP/2 conserva la semántica HTTP, pero cambia cómo viajan los mensajes. En vez de líneas de texto como las anteriores, usa **frames binarios**.

Sus ideas clave son:

- multiplexar varias peticiones/respuestas sobre una conexión, sin esperar a terminar una respuesta para iniciar otra;
- evitar bloqueos a nivel HTTP entre intercambios, aunque una pérdida de datos en TCP todavía puede retrasar todos los flujos de esa conexión;
- comprimir campos de cabecera repetidos.

El método sigue siendo `GET`, el estado sigue siendo `200`, y `Content-Type` sigue indicando el tipo de contenido. Lo que cambia es el transporte.

HTTP/3

HTTP/3 mantiene la semántica HTTP y usa QUIC sobre UDP. QUIC aporta conexión segura, control de entrega y flujos independientes.

No significa que HTTP/3 sea «poco fiable» por usar UDP. UDP por sí solo no garantiza entrega ni orden, pero QUIC añade las funciones que HTTP necesita.

Resumen práctico

- **HTTP/1.1:** útil para aprender porque los ejemplos son legibles como texto.
- **HTTP/2:** misma semántica, transporte binario y multiplexación sobre TCP.
- **HTTP/3:** misma semántica, transporte sobre QUIC/UDP y mejor gestión de flujos.

Para esta unidad basta con distinguir **qué conserva HTTP** y **qué cambia su transporte**.

Vocabulario mínimo

TCP

TCP transporta una secuencia de bytes de forma fiable y en orden entre dos extremos. Si faltan bytes, los recupera antes de entregar los siguientes en esa misma secuencia.

HTTP/1.1 y HTTP/2 suelen usar TCP.

UDP

UDP envía datagramas. Por sí solo no garantiza entrega, orden ni retransmisión. Eso no lo hace inútil: sirve como base para protocolos que añaden sus propias garantías.

QUIC

QUIC se construye sobre UDP y añade lo que HTTP/3 necesita: conexión segura, control de entrega, control de congestión y varios flujos.

La ventaja práctica es que la pérdida de datos en un flujo no tiene por qué bloquear la entrega de otro flujo independiente.

Frame binario

Un frame binario es una pieza estructurada del mensaje, con campos que procesa el programa. No es una línea de texto legible como en HTTP/1.1.

HTTPie o el navegador pueden mostrar los datos interpretados aunque en la red se hayan transmitido como frames.

Multiplexación

Multiplexar es alternar varias conversaciones lógicas en una misma conexión. Por ejemplo, una página puede solicitar HTML, CSS, JavaScript e imágenes sin esperar a que cada respuesta termine completamente para iniciar la siguiente.

No significa «multiplexar cabeceras». Tampoco significa que no haya límites: todos los flujos comparten red, CPU y memoria.

Compresión de campos

La compresión de campos codifica cabeceras repetidas (`Host`, `Content-Type`, `Accept`...) de forma más compacta. Es distinta de comprimir el cuerpo HTML o JSON.

Estructura de una petición

Una petición contiene línea inicial, cabeceras, una línea en blanco y contenido opcional.

```
GET /api/hello HTTP/1.1
Host: localhost:8080
Accept: application/json
```

Lectura del ejemplo:

- `GET`: método. El cliente pide una representación.
- `/api/hello`: destino dentro del servidor.

- `HTTP/1.1`: versión usada en la línea de ejemplo.
- `Host`: servidor lógico al que se dirige la petición.
- `Accept`: el cliente prefiere recibir JSON.
- Línea en blanco: indica que terminan las cabeceras.
- No hay cuerpo: un `GET` habitual no necesita enviar contenido.

Petición con cuerpo

Un `POST` suele llevar cuerpo:

```
POST /api/tasks HTTP/1.1
Host: localhost:8080
Content-Type: application/json
Accept: application/json

{"title":"Repasar HTTP"}
```

Lectura:

- `Content-Type: application/json` describe el cuerpo enviado.
- `Accept: application/json` expresa qué respuesta prefiere el cliente.
- El cuerpo JSON aparece después de la línea en blanco.

Estructura de una respuesta

Una respuesta contiene línea de estado, cabeceras, línea en blanco y cuerpo opcional.

```
HTTP/1.1 200 OK
Content-Type: application/json

{"message":"Hola desde el servidor"}
```

Lectura del ejemplo:

- `HTTP/1.1`: versión del mensaje mostrado.
- `200 OK`: estado de éxito.
- `Content-Type: application/json`: el cuerpo debe interpretarse como JSON.
- Línea en blanco: separa cabeceras y cuerpo.
- Cuerpo: representación devuelta por el servidor.

Respuesta sin cuerpo

No todas las respuestas llevan contenido. Por ejemplo, un borrado correcto puede responder:

```
HTTP/1.1 204 No Content
```

`204` significa éxito sin cuerpo. Si aparece un JSON después de `204`, algo está mal planteado.

Respuesta con recurso creado

Cuando se crea un recurso, la respuesta puede incluir `Location`:

```
HTTP/1.1 201 Created
Location: /api/tasks/7
Content-Type: application/json

{"id":7,"title":"Repasar HTTP"}
```

El estado comunica creación, `Location` indica dónde encontrar el recurso nuevo y el cuerpo puede incluir su representación.

Qué ve el navegador y qué viaja realmente

El navegador simplifica mucho la conversación. Al usuario le muestra una página, pero en segundo plano hay peticiones, respuestas, cabeceras, estados y cuerpos.

En las herramientas de desarrollo, pestaña **Red**, se pueden observar:

1. URL solicitada.
2. Método.
3. Estado.
4. Cabeceras de petición.
5. Cabeceras de respuesta.
6. Cuerpo o vista previa de la respuesta.

HTTPie muestra lo mismo desde terminal y es más cómodo para comparar evidencias.

Errores frecuentes al leer mensajes HTTP

- Mirar solo el cuerpo y olvidar el estado.
- Pensar que `200` siempre significa que la operación de negocio salió bien.
- Confundir `Accept` con `Content-Type`.
- Creer que HTTP/2 o HTTP/3 cambian el significado de métodos y estados.
- Pensar que HTTP es siempre «texto» porque HTTP/1.1 se puede explicar así.
- No localizar la línea en blanco que separa cabeceras y cuerpo.

Exploración local

Con Hello Server iniciado:

```
http --verbose GET http://localhost:8080/api/hello
```

Localiza en la salida:

1. Método.
2. Destino.
3. Cabeceras enviadas.
4. Estado de respuesta.
5. Cabeceras recibidas.
6. Cuerpo de respuesta.

Después abre la misma ruta en el navegador y busca esos elementos en **F12 → Red**.

Comprobación

Responde con tus palabras:

1. ¿Qué partes tiene una petición HTTP?
2. ¿Qué partes tiene una respuesta HTTP?
3. ¿Por qué `Content-Type` no es lo mismo que `Accept`?
4. ¿Qué se conserva al pasar de HTTP/1.1 a HTTP/2 o HTTP/3?
5. ¿Por qué HTTP/3 no es «poco fiable» aunque use UDP?
6. ¿Qué puedes saber mirando solo el estado `200 OK` y qué necesitas mirar para saber si la respuesta es HTML o JSON?

Esta evidencia prepara los documentos siguientes: métodos, estados, cabeceras y query string.